COMP 3920 Database Systems 1

Amazon Web Services (AWS) Relational Database Service (RDS) Replication

Introduction:

The goal of this document is to expand upon our existing knowledge using and managing Amazon's Relational Database Services (AWS RDS).

With Amazon's RDS Database instances, all of the management of the computer's hardware, operating system and software (aside from the few MySQL settings you customize) are handled completely by Amazon. In may ways, this is a good thing. This means you don't need to worry about all those details and you can focus on what's important – creating your database and applications. However, we don't have access to the file structure so we can't upload file to RDS and then import them. We'll need another way to import data.

Replication is a little different in Amazon's RDS. Many of the tasks required to setup or manage Replication in MySQL require a super user account with many privileges to the database. Our `masteruser`, that is added during the creation of the database instance, doesn't always have all those privileges. Instead what Amazon does is provide us with specific Replication stored procedures to help perform common replication tasks. We will use some of these procedures to setup a Master/Slave Replication pair on Amazon's RDS, which Amazon likes to call creating Read Replicas.

We will also look at how to handle Replication Failover in Amazon's RDS. Again, here the naming is a bit different, Replication Failover is referred to Promoting a Read Replica.

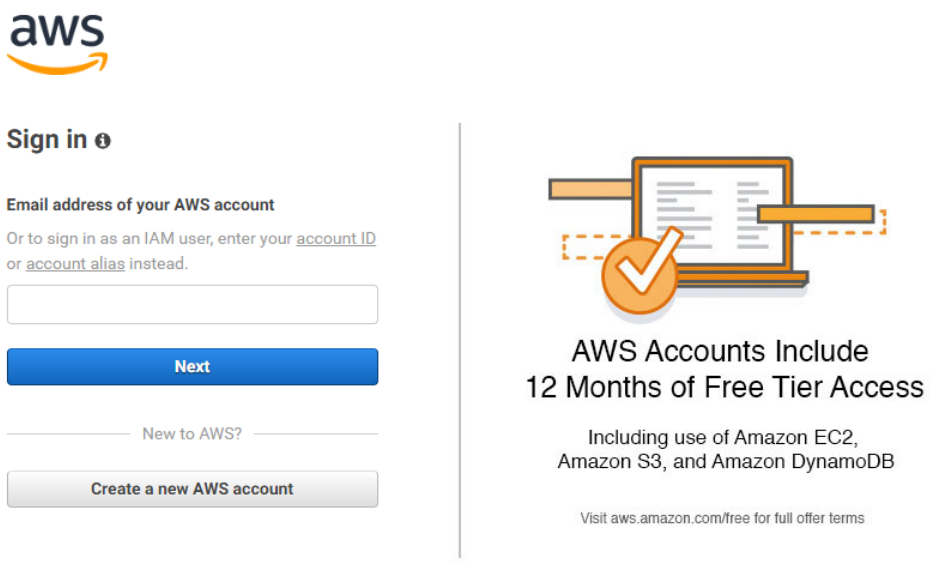
In this lab we will:

- Import external data into AWS RDS.
- Create an RDS Read Replica (Replication Slave).
- Promote a Read Replica to Master (Replication Failover).

Step 1:
Sign in to AWS.

Go to: https://console.aws.amazon.com

Enter your credentials to sign in.



Step 2:
Create a MySQL RDS Database Instance with the following settings:
(refer to last week's assignment if you need further details)

- **Engine:** MySQL – **8.0** or higher
  (restrict to Free Tier)
- **Instance Name:** comp3920-server-1
- **Username:** masteruser
- **Password: <your master password>**
- **Backup Window:** 10 days, No Maintenance Window Preference
- **Logs:** Select all that apply
- **Maintenance:** Enable auto upgrade, No Maintenance Window Preference
- **Deletion Protection:** Off

Step 3:
Import data into our new RDS instance.

Amazon doesn't give us access to the file structure of our RDS instance, so we can't just upload a file and import it like we have done so in the past. Instead if we have access to another computer (instance or VM) that we *can* upload a file to and that has the mysql client application, we can use this computer to import the data.

Download the file: `canadian_addresses.sql` from the Learning Hub on to a computer that has the `mysql` client.

Connect to your RDS instance and import the data using file redirection.

```
user@user-VirtualBox:~$ mysql -u masteruser -p -h acit4910-server-1.[redacted].rds.a
mazonaws.com < /[redacted]/canadian_addresses.sql
Enter password:
```

There are quite a few addresses in the database so give it some time.

Step 4:
Confirm that the database was uploaded successfully.

Show the number of rows in `places.address`;

Submit your screenshot:
Your screenshot must show the connection command **and** the number of rows in `places.address`.

Submit

06_import_successful.jpg

```
user@user-VirtualBox:~$ mysql -u masteruser -p -h acit4910-server-1.[redacted].r
ds.amazonaws.com
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 28
Server version: 8.0.13 Source distribution

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

masteruser@acit4910-server-1.[redacted].rds.amazonaws.com:[(none)] on Server 2>
select count(*) from places.address;
+----------+
| count(*) |
+----------+
| [redacted] |
+----------+
1 row in set (0.10 sec)

masteruser@acit4910-server-1.[redacted].rds.amazonaws.com:[(none)] on Server 2>
```

Step 5:
What privileges do you have on your `masteruser` on your RDS instance?

Are the privileges on the `masteruser` the same as `root` on a MySQL Server?

From your RDS instance (logged in as `masteruser`):

```
SHOW GRANTS;
```

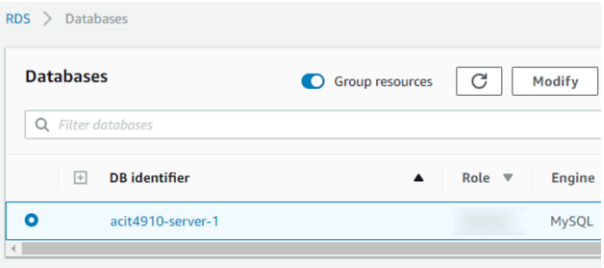
From your local MySQL Server (instance or VM) (logged in as `root`):

```
SHOW GRANTS;
```

Compare the privileges between `masteruser` on an RDS instance
and `root` on a local MySQL Server.

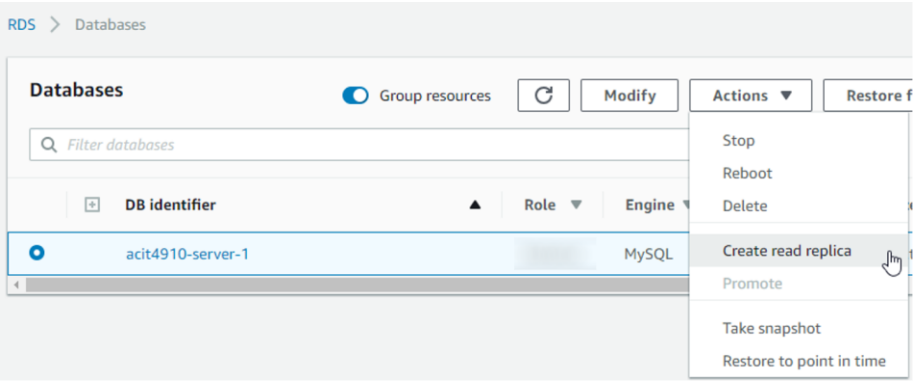
Step 6:
Create an RDS Read Replica.

Before we create a read replica let's take a look at our `comp3920-server-1`. In the Databases window, each DB Instance will have columns describing it. What do you see for the 'Role' of our `comp3920-server-1`?



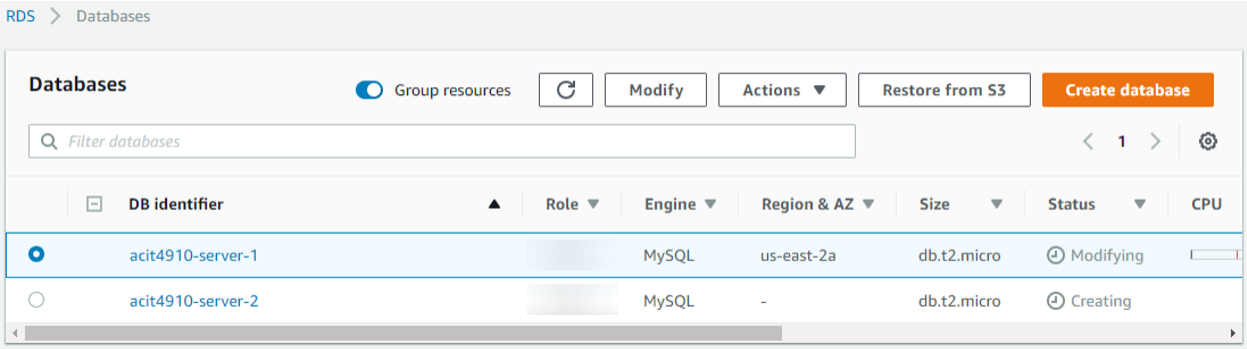
What is the **Role** description of our DB Instance `comp3920-server-1` *before* Replication?

Select your `comp3920-server-1` instance and click "Actions" and then "Create Read Replica".



Most of these settings for creating a read replica are the same for creating a brand new instance so we can leave many of them with the defaults, *except*:

- **Instance Name:** comp3920-server-2
- **Logs:** Select all that apply
- **Maintenance:** Enable auto upgrade



What is the **Role** description of our DB Instance `comp3920-server-1` *after* Replication?
What is the **Role** description of our DB Instance `comp3920-server-2`?

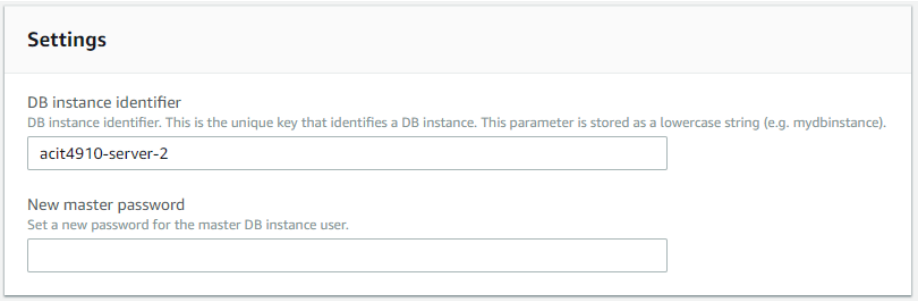
Step 7:
Confirm that your Read Replica (Slave) is replicating properly.

Create a new database on the *master*:
```
CREATE DATABASE replication_test;
```

Confirm replication is working on the *read replica (slave)*:
```
SHOW SLAVE STATUS \G
```

If you are having trouble logging in to the read replica with the `masteruser`, try resetting the password.

To reset the `masteruser` password, select your `comp3920-server-2` instance and click modify. Find the Settings Section and update the master user password.



Submit your screenshot:

Submit

07_read_replica_status_1.jpg

```
masteruser@acit4910-server-2.[redacted].rds.amazonaws.com:[(none)] on Server 2> sho
w slave status \G
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: [redacted]
                  Master_User: [redacted]
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: [redacted]
          Read_Master_Log_Pos: 532
               Relay_Log_File: relaylog.000055
                Relay_Log_Pos: 652
        Relay_Master_Log_File: [redacted]
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
          Replicate_Ignore_DB:
           Replicate_Do_Table:
       Replicate_Ignore_Table: innodb_memcache.cache_policies,innodb_memcache.config_options,my
sql.plugin,mysql.rds_configuration,mysql.rds_history,mysql.rds_monitor,mysql.rds_replication_st
atus,mysql.rds_sysinfo
      Replicate_Wild_Do_Table:
  Replicate_Wild_Ignore_Table:
                   Last_Errno: 0
                   Last_Error:
                 Skip_Counter: 0
          Exec_Master_Log_Pos: 532
              Relay_Log_Space: 1087
              Until_Condition: None
               Until_Log_File:
                Until_Log_Pos: 0
           Master_SSL_Allowed: No
           Master_SSL_CA_File:
           Master_SSL_CA_Path:
              Master_SSL_Cert:
            Master_SSL_Cipher:
               Master_SSL_Key:
        Seconds_Behind_Master: 0
Master_SSL_Verify_Server_Cert: No
                Last_IO_Errno: 0
```

Submit your screenshot:

Submit

08_read_replica_status_2.jpg

```
                   Last_Error:
                  Skip_Counter: 0
          Exec_Master_Log_Pos: 532
              Relay_Log_Space: 1087
              Until_Condition: None
               Until_Log_File:
                Until_Log_Pos: 0
           Master_SSL_Allowed: No
           Master_SSL_CA_File:
           Master_SSL_CA_Path:
              Master_SSL_Cert:
            Master_SSL_Cipher:
               Master_SSL_Key:
        Seconds_Behind_Master: 0
Master_SSL_Verify_Server_Cert: No
                Last_IO_Errno: 0
                Last_IO_Error:
               Last_SQL_Errno: 0
               Last_SQL_Error:
  Replicate_Ignore_Server_Ids:
             Master_Server_Id: 4[redacted]
                  Master_UUID: 5[redacted]
             Master_Info_File: mysql.slave_master_info
                    SQL_Delay: 0
          SQL_Remaining_Delay: NULL
      Slave_SQL_Running_State: Slave has read all relay log; waiting for more updates
           Master_Retry_Count: 86400
                  Master_Bind:
      Last_IO_Error_Timestamp:
     Last_SQL_Error_Timestamp:
               Master_SSL_Crl:
           Master_SSL_Crlpath:
           Retrieved_Gtid_Set:
            Executed_Gtid_Set:
                Auto_Position: 0
         Replicate_Rewrite_DB:
                 Channel_Name:
           Master_TLS_Version:
       Master_public_key_path:
        Get_master_public_key: 0
1 row in set (0.09 sec)

masteruser@acit4910-server-2.[redacted].rds.amazonaws.com:[(none)] on Server 2>
```

Confirm replication of the existing `places.address` table worked and the `replication_test` database replicated properly on the *read replica (slave)*:

```
SELECT COUNT(*) FROM places.address;
SHOW DATABASES;
```

Submit your screenshot:
Your screenshot must show the `places.address` row count **and** `replication_test` in the `SHOW DATABASES` list.

Submit

09_replication_successful.jpg

```
masteruser@acit4910-server-2.[redacted].rds.amazonaws.com:[(none)] on Server 2> sho
w databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| places             |
| replication_test   |
| sys                |
+--------------------+
6 rows in set (0.09 sec)

masteruser@acit4910-server-2.[redacted]rds.amazonaws.com:[(none)] on Server 2> sel
ect count(*) from places.address;
+----------+
| count(*) |
+----------+
|[redacted]|
+----------+
1 row in set (0.11 sec)

masteruser@acit4910-server-2.[redacted]rds.amazonaws.com:[(none)] on Server 2>
```

Step 8:
Determine Amazon RDS's Replication Method.

Run the commands necessary to determine Amazon RDS's replication method.

By default, does Amazon RDS use Binlog Replication or GTID Replication?

Step 9:
Determine Amazon RDS's Replication Data Format.

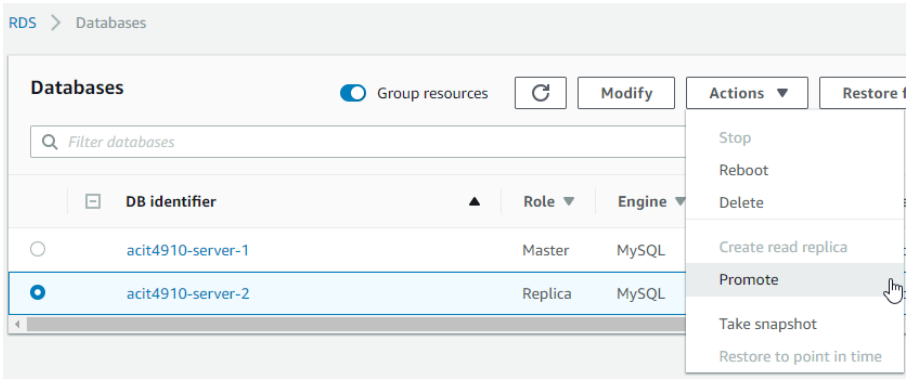
Run the commands necessary to determine Amazon RDS's replication method.

```
SELECT @@binlog_format;
```

By default, which Replication Data Format does Amazon RDS use?
(Statement Based, Row Based or Mixed).

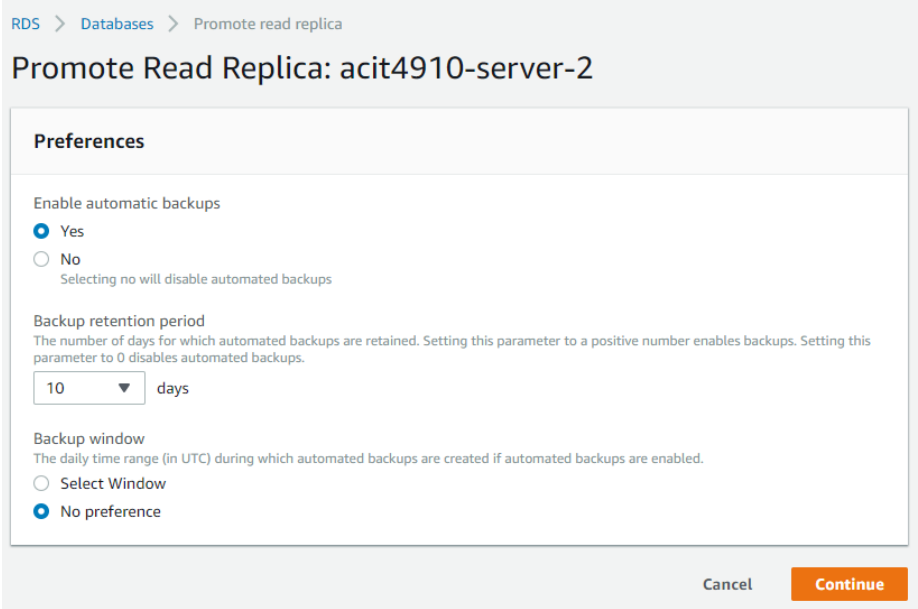
Step 10:
Promote your Read Replica to a Master.

Select your `comp3920-server-2` RDS instance and click Actions, then click Promote.

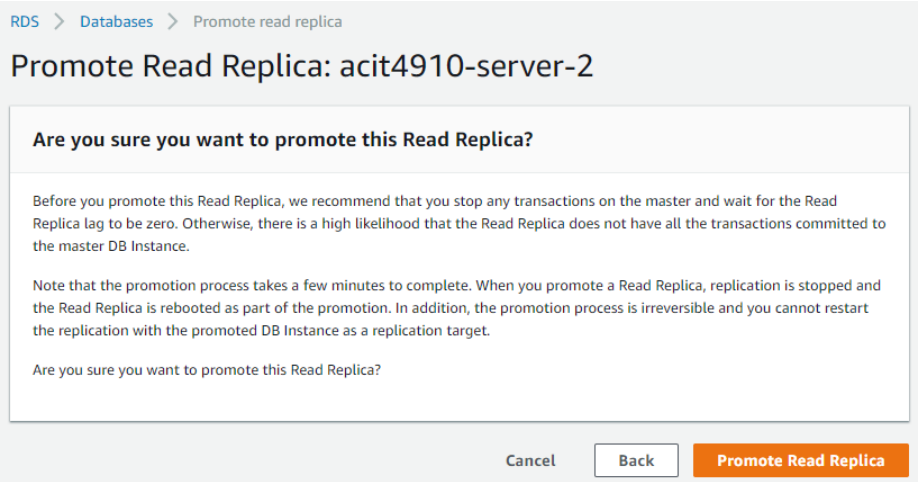


Set your Backup Retention time to 10 days, no Maintenance Window Preference.

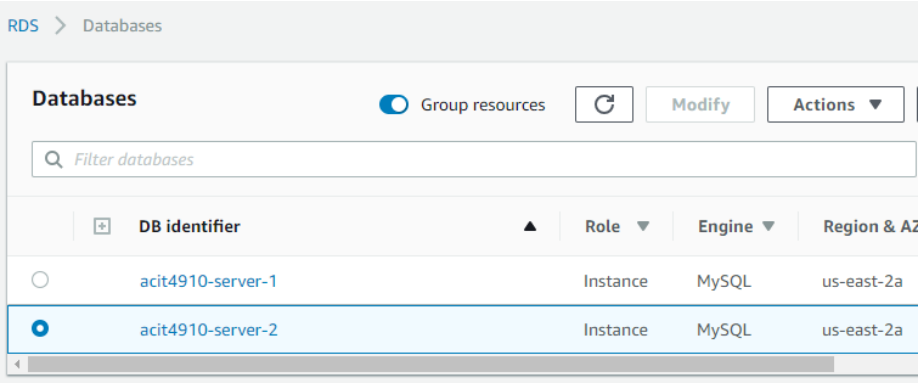
Click Continue.



Pay attention to the information on the warnings screen (this will help answer the lab questions below):



Once complete your Read Replica will be a standalone RDS instance:



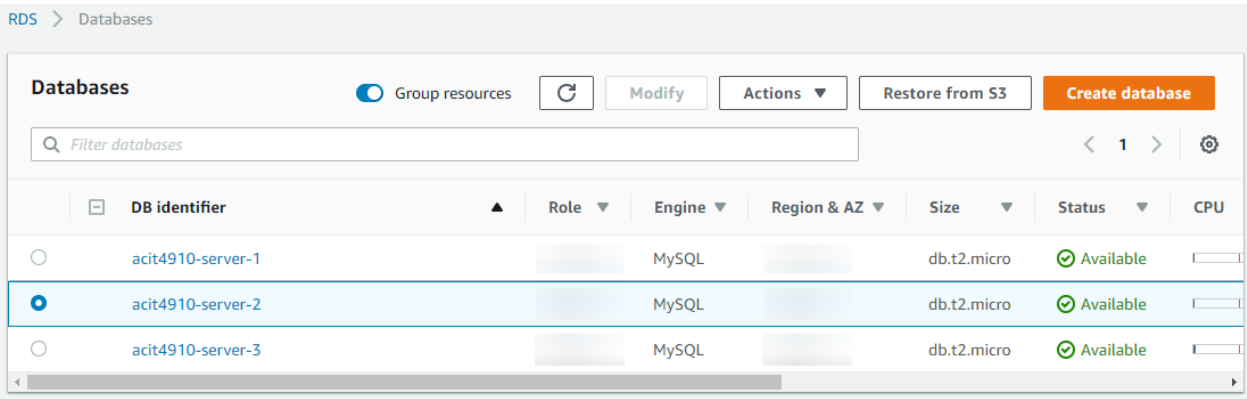
To re-establish a Master/Slave Replication Pair, create a Read Replica of `comp3920-server-2`. Call your new read replica `comp3920-server-3`. Wait for the replica to be **Available**.

What do we need to know before we can safely promote a read replica?

Submit your screenshot:
Your screenshot must show your 3 instances with an **Available** status.

Submit

10_three_instances.jpg



Step 11:
Setup GTID Replication on `comp3920-server-2` and `comp3920-server-3`.

Amazon's Reference provides us with a thorough example of everything required to switch from Binlog Replication to GTID Replication on existing Master/Read Replica Pairs

Amazon's Reference: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/mysql-replication-gtid.html
See section on: *Configuring GTID-Based Replication for Existing Read Replicas*

Step 1 is to ensure that your MySQL Versions are both >= 5.7.23 (but not 8.0). Amazon RDS doesn't support GTIDs on MySQL 8.0 yet.

Step 2 is optional. We are not going to use this step.

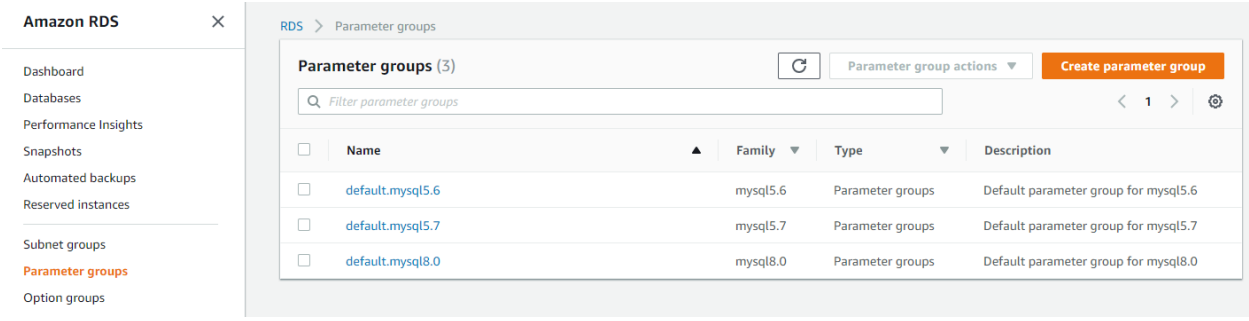
Steps 3 to 4 are for if you are currently using your replication server pair for on-going traffic. We would want to ensure we don't lose any transactions during the Binlog to GTID swap over process. Since there is no traffic to our database servers at the moment we are going to skip these steps as well.

Step 5. Reset the GTID parameters for GTID-based replication only.
Remember from our Lab on GTID Replication, we set 2 parameters in the my.ini/my.cnf configuration files for GTID Replication:

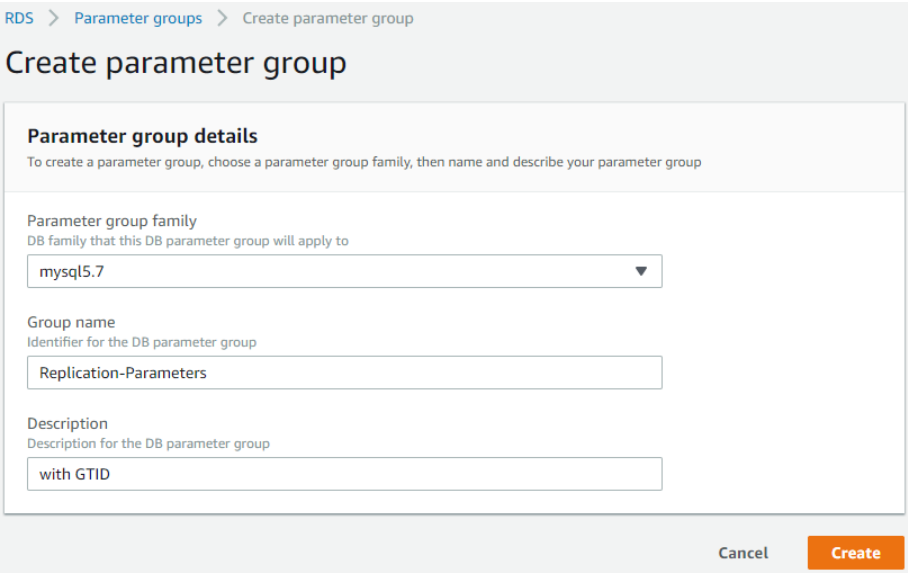
```
gtid-mode=ON
enforce-gtid-consistency=ON
```

We need to do the same here in Amazon's RDS, however, we don't have access to the file system or to the configuration files directly. Instead Amazon RDS provides access to the mysql configuration through, what they call, *Parameter Groups*.

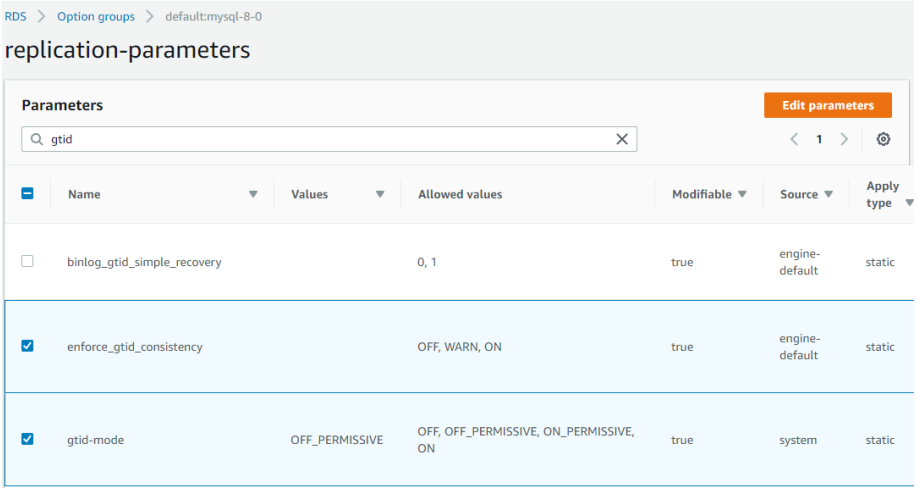
We are going to create a new parameter group for GTID Replication and give our instances this new parameter group.
Click on Parameter Groups on the left menu (under Amazon RDS),



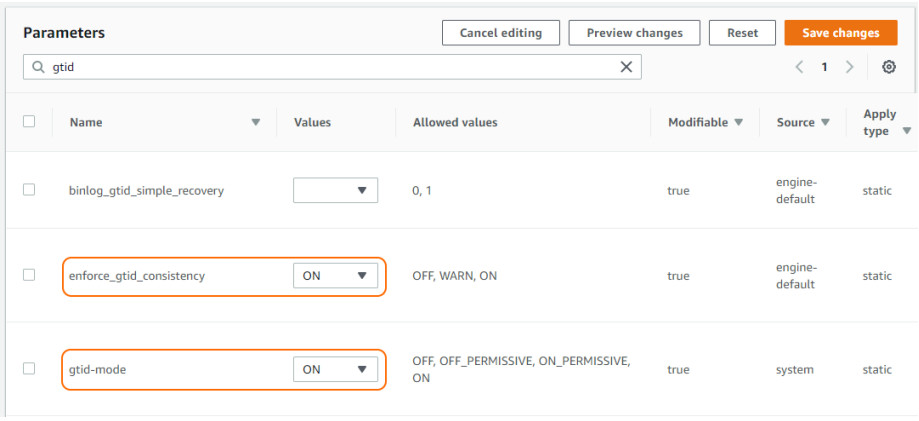
Give your parameter group the name `Replication-Parameters`, with a description of `with GTID`.



Search for `GTID`, Click "Edit Parameters".

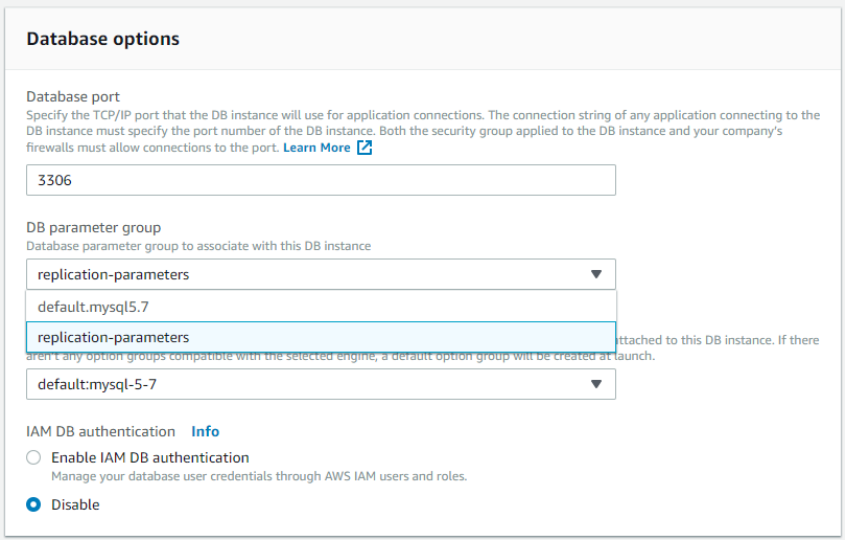


Change the values for `gtid-mode` and `enforce-gtid-consistency` to `ON`.

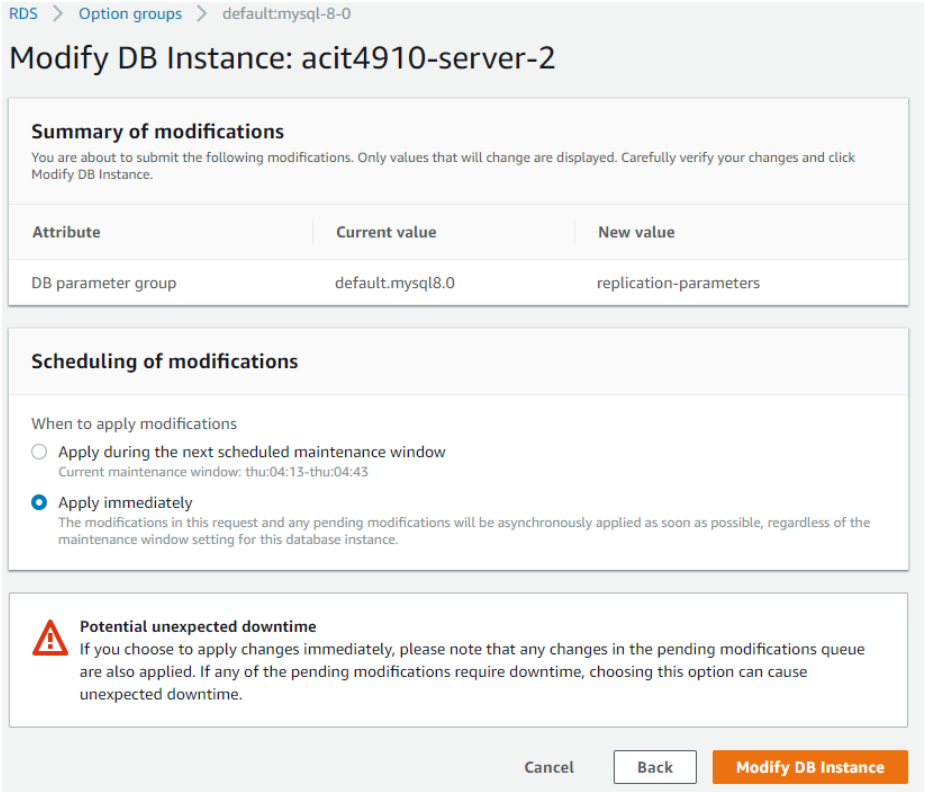


Click "Save changes".

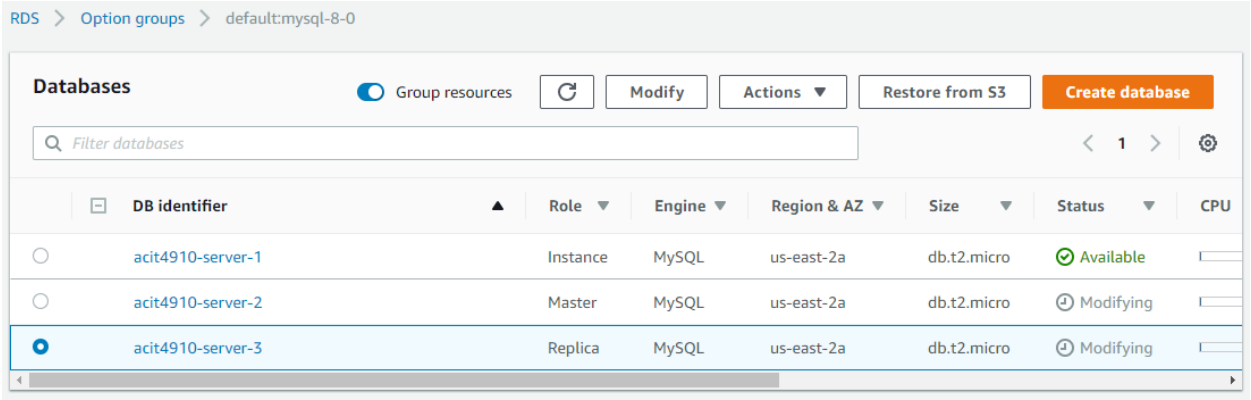
For both `comp3920-server-2` and `comp3920-server-3` instances, change the parameter group to `Replication-Parameters`.



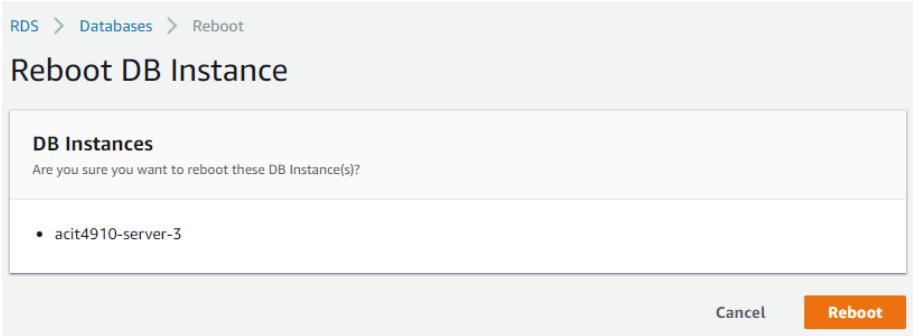
Select Apply immediately, and click "Modify DB Instance".



Wait a bit.



Reboot both instances.
Select the instances (one at a time), Click "Actions", then click "Reboot".
You may need to reboot your Read Replica ***first***, then your Master.



Confirm that the configuration changes have taken effect by running the following command on the *Read Replica*:

```
SHOW VARIABLES LIKE '%gtid%';
```

Variable_name	Value
binlog_gtid_simple_recovery	ON
enforce_gtid_consistency	ON
gtid_executed_compression_period	1000
gtid_mode	ON
gtid_next	AUTOMATIC
gtid_owned	
gtid_purged	a8eec70a-4dc8-11e9-89e7-0a4c9936901a:1
session_track_gtids	OFF

Next, we need to set the Master Auto Position.
On a local MySQL Server (non-AWS RDS Instance) we could use the `CHANGE MASTER TO MASTER_AUTO_POSITION=1` command. However on Amazon's RDS Instances we don't have privileges to do this. Instead Amazon has provided us with a stored procedure to this for us.

For a list of stored procedures provided by Amazon go here:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Appendix.MySQL.SQLRef.html

The one we need is: mysql.rds_set_master_auto_position();

Run this command:

```
CALL mysql.rds_set_master_auto_position(1);
```

Submit your screenshot:
Your screenshot should show the `CALL mysql.rds_set_master_auto_position(1);` and its results.

11_GTID_Auto_Position.jpg

Submit

```
masteruser@acit4910-server-3.[redacted].rds.amazonaws.com:[(none)] on Server 2>
CALL mysql.rds_set_master_auto_position(1);
+----------------------------------------+
| Message                                |
+----------------------------------------+
| Master Auto Position [redacted]        |
+----------------------------------------+
1 row in set (0.10 sec)

+---------------------------+
| Message                   |
+---------------------------+
| [redacted]                |
+---------------------------+
1 row in set (2.11 sec)

Query OK, 0 rows affected (2.11 sec)
```

How do we access Amazon's RDS Configuration file (my.ini/my.cnf)?
How do we configure replication settings such as GTID Auto Position?

IMPORTANT: Clean up your AWS Instances when you are done the lab!

Even though we created our instances as db.t2.micro (Free Tier), we should clean up instances just in case. Also, since we don't need our snapshots, we should delete those as well. The storage that is required to keep our snapshots may end up costing us money as well.